

Homework 6: Software Testing and Built-in Modules

- Due Sunday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 1 at 12am - Jul 17 at 11:59pm

Learning Objectives

- Build larger programs with multiple functions.
- Write unit tests.
- Use math and random modules.

Things to consider

1. Submit the solutions to all the problems on gradescope. Files to be submitted are: Hw06.py, Hw06_tests.py and Reflection.pdf. Submission of pdf files is not enforced in the grader.
2. The output of your program must match the examples exactly for points.
3. This is an individual assignment. Do not use AI or ask your friends for solutions. We will be running similarity checks in gradescope.
4. Run given tests locally before submitting. It will save you a lot of time. Local runs are a lot faster than gradescope submissions.

Q1: Converting a number to words.

Converting a dollar amount to an amount in words is a common operation in consumer banking. For example, a check amount 1356 is also written as “One Thousand Three Hundred Fifty Six”. For this question, you will be writing multiple functions, with the goal of converting any amount up to 999 billion dollars into words.

Part A

 Create a global variable named **amounts** which is a dictionary containing the following values:
(key: value)

- 1: 'One'
- 2: 'Two'
- 3: 'Three'
- 4: 'Four'

5: 'Five'

6: 'Six'

7: 'Seven'

8: 'Eight'

9: 'Nine'

10: 'Ten'

11: 'Eleven'

12: 'Twelve'

13: 'Thirteen'

14: 'Fourteen'

15: 'Fifteen'

16: 'Sixteen'

17: 'Seventeen'

18: 'Eighteen'

19: 'Nineteen'

20: 'Twenty'

30: 'Thirteen'

40: 'Forty'

50: 'Fifty'

60: 'Sixty'

70: 'Seventy'

80: 'Eighty'



90: 'Ninety'

Part B

Write a function named `convertTwoDigit` which accepts a two digit number and returns a string representation of the value. Return empty string "" if the number is 0.

The task can be completed by looking up the dictionary if the input is less than or equal to 20. If the input is greater than 20, splitting the number the number into 10's digit and 1's digit, followed by dictionary lookups to create the string will be useful.

Example (This is not a comprehensive set of tests):

```
number = 57
out = convertTwoDigit(number)      # out must be 'Fifty Seven'
```

Part C

Write a function named `convertThreeDigit` which accepts a three digit number and returns a string representation of the value. Return empty string "" if the number is 0.

Use call to `convertTwoDigit` function to help you in the process.

Example (This is not a comprehensive set of tests):

```
number = 571
out = convertThreeDigit(number)      # out must be 'Five Hundred Seventy One'
number = 600
out = convertThreeDigit(number)      # out must be 'Six Hundred'
number = 61
out = convertThreeDigit(number)      # out must be 'Sixty One'
```

Note that the word 'Hundred' is added for three digit case but not for two digit case.

Part D

Write a function named `amountInWords` which accepts a number and returns a string representation of the value. Return empty string "" if the number is 0. You need to separate the number into three digit chunks. Use `convertThreeDigit` to help with 3 digit conversions. Then join the chunks together with the magnitude of each chunk such as 'Billion', 'Million' and 'Thousand'.

Example (This is not a comprehensive set of tests):

```
number = 57167832567
out = amountInWords(number)          # out must be 'Fifty Seven Billion One Hundred Sixty Seven Million E
ight
# Hundred Thirty Two Thousand Five Hundred Sixty Seven'
number = 61
out = amountInWords(number)          # out must be 'Sixty One'
```

Part E

In the file, `Hw06_tests.py`, write a to test each function.

- `convertTwoDigit_test()`
- `convertThreeDigit_test()`
- `amountInWords_test()`

You can import your functions from `Hw06.py` to run tests on them. But before submitting to gradescope, change the import to the line given below:

```
from Hw06_Instrumented import amountInWords , convertTwoDigit, convertThreeDigit    # this is needed for gradescope tests.
```

Each of these functions should test multiple cases. You must include the expected output and Pass/Fail for each test as well. Writing additional functions to help with the process is encouraged. Autograder will evaluate your test functions for code coverage and manual grading will be done to ensure that expected output and checks are included in the test set.

Question 2: Stock Price Trajectory Risk Assessment

In this question, we will be stock traders and will try to estimate the probability of a stock price reaching a certain threshold in a given number of days. For example: what is the probability of a SpaceX share, currently valued at \$150, of reaching \$160 in 30 days. We will do this for a full portfolio of stocks. You will write a function called `simulate_stock_risk(portfolio, risk_free_rate, days, num_trials)`.

- `portfolio` is a list of dictionaries. Each dictionary represents a stock asset being evaluated, containing the keys: `"ticker"` (string representing the stock name), `"current_price"` (float), `"volatility"` (float, annual standard deviation in price changes), and `"strike_price"` (float, target threshold price for which we are calculating probability).
- `risk_free_rate` is a float representing the annual constant risk-free rate of return (e.g., `0.05` for 5%). This is usually the U.S. 10-Year Treasury Note.
- `days` is an integer representing number of day into the future we will be estimating stock pricing.
- `num_trials` is an integer represents the total number of times we will simulate for each stock asset.

Your function must evaluate the stock of each stock asset by running a Monte Carlo random walk simulation to determine the probability that the asset's price will finish strictly above its designated target `strike_price`. However, it must follow these strict rules:

1. If `num_trials` or `days` is less than or equal to 0, or if the `portfolio` list is empty, the function must immediately return an empty dictionary `{}`.

2. To simulate daily price movements, annual parameters must first be scaled to a daily time-increment (dt), assuming 252 trading days in a year:

$$dt = \frac{1}{252}$$

3. For each individual stock in the portfolio, you must run `num_trials` independent simulations. Each simulation starts at the asset's `"current_price"` (S_0) and iteratively paths forward day-by-day for the specified number of `days`. The price on day t (S_t) is updated from the previous day's price (S_{t-1}) using a geometric random walk step formula:

$$S_t = S_{t-1} \cdot e^{\left(r - \frac{\sigma^2}{2}\right)dt + \sigma \cdot Z \cdot \sqrt{dt}}$$

Where r is the `risk_free_rate`, σ is the asset's `"volatility"`, and Z is a random sample drawn from a standard normal distribution (mean of 0 and standard deviation of 1). Use the function `random.gauss()` for these numbers.

4. A simulation trial is counted as a ****success**** if the final calculated stock price on the last day (S_{days}) is strictly greater than the asset's assigned `"strike_price"`.

The function must return a dictionary where the keys are the stock `"ticker"` strings, and the values are the calculated probabilities that the price will reach target as floats rounded to 4 decimal places.

$$\text{Probability} = \frac{\text{Success Trials}}{\text{Total Trials}}$$

Example 1: Normal simulation case

```
stocks = [
    {"ticker": "AAPL", "current_price": 150.0, "volatility": 0.25, "strike_price": 300.0}
    {"ticker": "MSFT", "current_price": 300.0, "volatility": 0.20, "strike_price": 302.0}
]

print(simulate_stock_risk(stocks, 0.05, 20, 50000))

# Possible valid answers:
# {"AAPL": 0.0, "MSFT": 0.4682}
# Note: AAPL should consistently output 0.0000 because it is mathematically impossible for a stock with
# 25% volatility to double in 20 trading days. MSFT's probability will fluctuate tightly around ~0.46 to
# 0.47 due to randomness.
```

Example 2: Empty/Invalid case

```
print(simulate_stock_risk([], 0.05, 252, 1000))

# Expected Output:
# {}
```

Validation Rules for Grading: We will run the homework submissions at least 10 times per test case to evaluate dynamic variance. Invalid or missing inputs must gracefully exit with an empty

dictionary.

Reflection (5% bonus points for submission)

Reflect on the topics covered this week and write short answers for the questions below. Submit as a pdf called Reflection.pdf.

1. What did we learn this week?
2. Rate your understanding of the topics this week. (1. Did not understand at all, ... 5. Fully understand the topic)
3. Are there any questions related to this week's topics? We will address the questions either at the beginning of a lab or through a post.

This tool needs to be loaded in a new browser window

Load Homework 6: Software Testing and Built-in Modules in a new window

